

"""

SmartZ-EDU — Hệ thống Z-score thích ứng bằng Mô hình Hỗn hợp Gauss Mềm (Soft GMM) hỗ trợ đánh giá công bằng và ra quyết định quản lý giáo dục THCS.

Phương pháp chính: Z-score GMM MỀM (soft/posterior-weighted) — mỗi học sinh được gán trọng số xác suất (gamma) thuộc về từng "cụm năng lực", thay vì bị ép buộc phân loại cứng vào một nhóm duy nhất.

Cơ chế thích ứng tự động (TỔNG QUÁT): hệ thống tự động dò số đỉnh (số thành phần Gauss) từ $k=1$ đến $k=K_MAX$, chọn k có BIC nhỏ nhất — không áp đặt sẵn "phải có đúng 2 đỉnh", mà để dữ liệu tự quyết định. Trên dữ liệu thực tế của trường (có đúng 2 loại hình lớp), thuật toán tự tìm ra $k=2$ là tối ưu — một bằng chứng khách quan củng cố giả thuyết ban đầu, thay vì một giả định được áp đặt trước.

Chạy local: streamlit run app.py

"""

```
import io
import numpy as np
import pandas as pd
import streamlit as st
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.mixture import GaussianMixture

st.set_page_config(
    page_title="SmartZ-EDU — Z-score thích ứng bằng Soft GMM",
    page_icon="🎓",
    layout="wide",
    initial_sidebar_state="expanded",
)
```

$K_MAX_DEFAULT = 4$ # số đỉnh tối đa mà hệ thống sẽ dò thử

```
# -----
# GIAO DIỆN: CSS TỰ CHỈNH
# -----
```

```
st.markdown(
    """
    <style>
    .main > div { padding-top: 1.2rem; }

    .smartz-hero {
        background: linear-gradient(135deg, #1e3a5f 0%, #2c5f8a 45%, #3a8fb7 100%);
```

```

border-radius: 18px;
padding: 2rem 2.2rem;
margin-bottom: 1.4rem;
color: white;
}
.smartz-hero h1 { font-size: 1.9rem; font-weight: 800; margin: 0 0 0.4rem 0; color: white; }
.smartz-hero p { font-size: 1rem; opacity: 0.92; margin: 0; max-width: 850px; line-height: 1.5; }
.smartz-badge {
  display: inline-block; background: rgba(255,255,255,0.18);
  border: 1px solid rgba(255,255,255,0.35); border-radius: 999px;
  padding: 0.15rem 0.75rem; font-size: 0.78rem; margin-right: 0.4rem; margin-top: 0.7rem;
}
.smartz-decision-gmm {
  background: linear-gradient(90deg, #e8f8ee 0%, #d6f2e0 100%);
  border-left: 5px solid #2e9e5b; border-radius: 10px; padding: 0.85rem 1.1rem; font-size:
0.95rem;
}
.smartz-decision-naive {
  background: linear-gradient(90deg, #eaf2fb 0%, #dcebf9 100%);
  border-left: 5px solid #3a7ec9; border-radius: 10px; padding: 0.85rem 1.1rem; font-size:
0.95rem;
}
.smartz-decision-multi {
  background: linear-gradient(90deg, #f5eefc 0%, #ece0f9 100%);
  border-left: 5px solid #8e5fc9; border-radius: 10px; padding: 0.85rem 1.1rem; font-size:
0.95rem;
}
.smartz-flag-up {
  background: #fff4e5; border-left: 5px solid #e6912c; border-radius: 10px;
padding: 0.7rem 1rem; margin-bottom: 0.5rem;
}
.smartz-flag-down {
  background: #fdeaea; border-left: 5px solid #d9534f; border-radius: 10px;
padding: 0.7rem 1rem; margin-bottom: 0.5rem;
}
div[data-testid="stMetric"] { background: rgba(120,120,120,0.06); border-radius: 12px;
padding: 0.7rem 0.9rem; }
</style>
"""
unsafe_allow_html=True,
)

```

LỖI THUẬT TOÁN (TỔNG QUÁT k THÀNH PHẦN)

```
# -----
```

```
def fit_gmm_k(x: np.ndarray, k: int):  
    """Fit GMM k thành phần, sắp xếp các thành phần theo trung bình tăng dần."""  
    gmm = GaussianMixture(n_components=k, n_init=20, random_state=42).fit(x.reshape(-1, 1))  
    means = gmm.means_.flatten()  
    stds = np.sqrt(gmm.covariances_.flatten())  
    weights = gmm.weights_.flatten()  
    order = np.argsort(means)  
    components = [(means[i], stds[i], weights[i]) for i in order]  
    return components, gmm.bic(x.reshape(-1, 1))
```

```
def adaptive_search(x: np.ndarray, k_max: int = K_MAX_DEFAULT):  
    """Tự động dò số đỉnh (thành phần Gauss) từ k=1 đến k_max, chọn k có BIC nhỏ nhất.  
    Trả về (best_k, components_tại_best_k, bang_bic {k: bic})."""  
    n = len(x)  
    k_max_eff = max(1, min(k_max, n // 5)) # tránh overfit khi mẫu nhỏ (tối thiểu ~5 điểm/thành phần)  
    bic_table, comps_by_k = {}, {}  
    for k in range(1, k_max_eff + 1):  
        comps, bic = fit_gmm_k(x, k)  
        bic_table[k] = bic  
        comps_by_k[k] = comps  
    best_k = min(bic_table, key=bic_table.get)  
    return best_k, comps_by_k[best_k], bic_table
```

```
def z_naive(x: np.ndarray) -> np.ndarray:  
    return (x - x.mean()) / x.std(ddof=1)
```

```
def z_soft_gmm_k(x: np.ndarray, components):  
    """Z-score GMM MỀM tổng quát cho k thành phần bất kỳ:  $Z^* = \sum_k \gamma_k \cdot z_k$ ,  
    với  $\gamma_k$  là xác suất hậu nghiệm (posterior) theo định lý Bayes."""  
    dens = np.zeros((len(components), len(x)))  
    for i, (mu, s, pi) in enumerate(components):  
        dens[i] = pi * stats.norm.pdf(x, mu, s)  
    denom = dens.sum(axis=0)  
    denom = np.where(denom <= 0, 1e-300, denom)  
    gammas = dens / denom # shape (k, n)  
    z_each = np.array([(x - mu) / s for (mu, s, _) in components]) # shape (k, n)  
    z_soft = (gammas * z_each).sum(axis=0)  
    return z_soft, gammas
```

```

def z_hard_gmm_k(x: np.ndarray, components, gammas):
    best_comp = np.argmax(gammas, axis=0)
    z_each = np.array([(x - mu) / s for (mu, s, _) in components])
    return z_each[best_comp, np.arange(len(x))]

def analyze_column(raw: pd.DataFrame, group_col: str, score_col: str, k_max: int):
    """Xử lý một cột điểm: làm sạch dữ liệu, tự động dò số đỉnh, tính các loại Z-score.
    Giữ nguyên chỉ số (index) gốc của `raw` để ghép nối chính xác giữa nhiều cột điểm."""
    data = raw[[group_col, score_col]].copy()
    data[score_col] = pd.to_numeric(data[score_col], errors="coerce")
    data = data.dropna() # giữ nguyên index gốc, KHÔNG reset_index
    if data.empty:
        return None

    x = data[score_col].values
    best_k, components, bic_table = adaptive_search(x, k_max=k_max)

    result = data.copy()
    result["Z_truyen_thong"] = z_naive(x).round(3)

    gammas = None
    if best_k >= 2:
        z_s, gammas = z_soft_gmm_k(x, components)
        z_h = z_hard_gmm_k(x, components, gammas)
        for i in range(best_k):
            result[f"gamma_cum_{i+1}"] = gammas[i].round(4)
        result["Z_GMM_mem"] = z_s.round(3)
        result["Z_GMM_cung"] = z_h.round(3)

    return {
        "score_col": score_col, "data": data, "x": x,
        "best_k": best_k, "components": components, "bic_table": bic_table,
        "gammas": gammas, "result": result,
    }

def render_distribution_chart(score_col: str, x: np.ndarray, components, best_k: int):
    fig, ax = plt.subplots(figsize=(9, 4.6))
    lo, hi = float(np.floor(x.min())), float(np.ceil(x.max()))
    bins = np.arange(max(0, lo - 0.5), hi + 1.0, 0.5)
    ax.hist(x, bins=bins, density=True, color="#8ecae6", edgecolor="white", alpha=0.85,

```

```

        label="Phổ điểm thực tế")
xs = np.linspace(bins[0], bins[-1], 400)
palette = ["#e76f51", "#2a9d8f", "#e9c46a", "#8e5fc9"]
if best_k >= 2:
    total = np.zeros_like(xs)
    for i, (mu, s, pi) in enumerate(components):
        comp_curve = pi * stats.norm.pdf(xs, mu, s)
        total += comp_curve
        ax.plot(xs, comp_curve, color=palette[i % len(palette)], ls="--", lw=2,
                label=f"Cụm {i+1} ( $\mu \approx \{\mu: .1f\}$ , tỉ trọng  $\{\pi: .2f\}$ )")
    ax.plot(xs, total, color="#1d3557", lw=2.4, label=f"Tổng hợp GMM {best_k} đỉnh")
else:
    mu, sd = x.mean(), x.std()
    ax.plot(xs, stats.norm.pdf(xs, mu, sd), color="#1d3557", lw=2.4,
            label=f"Phân phối chuẩn ( $\mu \approx \{\mu: .1f\}$ )")
ax.set_title(f"Phổ điểm {score_col} — bước nhảy 0.5 điểm", fontsize=12, fontweight="bold")
ax.set_xlabel("Điểm số"); ax.set_ylabel("Mật độ xác suất")
ax.set_xticks(np.arange(bins[0], bins[-1] + 0.5, 0.5))
ax.tick_params(axis="x", rotation=45)
ax.legend(fontsize=8, frameon=False)
ax.grid(True, ls="--", alpha=0.35)
ax.spines[["top", "right"]].set_visible(False)
plt.tight_layout()
return fig

```

```

def render_bic_chart(bic_table: dict, best_k: int):
    fig, ax = plt.subplots(figsize=(5, 3.2))
    ks = sorted(bic_table.keys())
    vals = [bic_table[k] for k in ks]
    colors = ["#2e9e5b" if k == best_k else "#a8b3bd" for k in ks]
    ax.bar([str(k) for k in ks], vals, color=colors)
    ax.set_xlabel("Số đỉnh giả định (k)"); ax.set_ylabel("BIC (càng thấp càng tốt)")
    ax.set_title("So sánh BIC theo số đỉnh", fontsize=10, fontweight="bold")
    for i, v in enumerate(vals):
        ax.text(i, v, f"{v:.0f}", ha="center", va="bottom", fontsize=8)
    ax.spines[["top", "right"]].set_visible(False)
    plt.tight_layout()
    return fig

```

```

def render_analysis(raw: pd.DataFrame, group_col: str, r: dict, exception_threshold: float):
    """Hiển thị toàn bộ kết quả phân tích cho MỘT cột điểm (dùng chung cho cả 2 chế độ)."""
    score_col, x, result = r["score_col"], r["x"], r["result"]

```

```

best_k = r["best_k"]

m1, m2, m3 = st.columns(3)
m1.metric("Số học sinh hợp lệ", f"{len(x)}")
m2.metric("Số đỉnh tự động phát hiện (k)", f"{best_k}")
m3.metric("BIC tại k tối ưu", f"{r['bic_table'][best_k]:.1f}")

if best_k == 1:
    st.markdown(
        '<div class="smartz-decision-naive">🔍 &nbsp;Hệ thống dò k = 1..%d và xác định '
        '<b>phổ điểm chỉ có 1 đỉnh</b> (đơn phương thức) → tự động dùng '
        '<b>Z-score truyền thống</b>.</div>' % max(r["bic_table"].keys()),
        unsafe_allow_html=True,
    )
elif best_k == 2:
    st.markdown(
        '<div class="smartz-decision-gmm">✅ &nbsp;Hệ thống tự động dò và xác định '
        '<b>phổ điểm có 2 đỉnh</b> là mô tả tối ưu (BIC nhỏ nhất) → áp dụng '
        '<b>Z-score GMM Mềm với 2 cụm</b>.</div>',
        unsafe_allow_html=True,
    )
else:
    st.markdown(
        f'<div class="smartz-decision-multi">🔍 &nbsp;Hệ thống tự động dò và xác định '
        f'<b>phổ điểm có {best_k} đỉnh</b> là mô tả tối ưu → áp dụng <b>Z-score GMM Mềm '
        f'với {best_k} cụm</b>. Lưu ý: với trên 2 cụm, hệ thống chưa tự ánh xạ từng cụm '
        f'sang đúng một loại hình lớp hành chính cụ thể.</div>',
        unsafe_allow_html=True,
    )

st.write("")
tab_names = ["📊 Biểu đồ phổ điểm", "🔬 Cơ chế thích ứng (BIC)", "📋 Bảng kết quả"]
if best_k == 2:
    tab_names.insert(2, "🚩 Ngoại lệ sự phạm")
elif best_k > 2:
    tab_names.insert(2, "🌿 Xác suất theo cụm")
tabs = st.tabs(tab_names)

with tabs[0]:
    fig = render_distribution_chart(score_col, x, r["components"], best_k)
    st.pyplot(fig, use_container_width=True)

with tabs[1]:
    cL, cR = st.columns([1, 1])

```

```

with cL:
    fig_bic = render_bic_chart(r["bic_table"], best_k)
    st.pyplot(fig_bic, use_container_width=True)
with cR:
    bic_df = pd.DataFrame(
        {"Số đỉnh (k)": list(r["bic_table"].keys()), "BIC": list(r["bic_table"].values())}
    )
    bic_df["Được chọn"] = bic_df["Số đỉnh (k)"].apply(lambda k: "✅" if k == best_k else "")
    st.dataframe(bic_df, use_container_width=True, hide_index=True)
    st.caption(
        "Hệ thống dò từ k=1 đến k tối đa, KHÔNG áp đặt sẵn số đỉnh — "
        "k được chọn là giá trị cho BIC nhỏ nhất."
    )

idx_extra = 2
if best_k == 2:
    with tabs[idx_extra]:
        st.caption(
            f"Học sinh được gán cờ khi xác suất  $\gamma$  thuộc **cụm khác với nhóm hành chính hiện "
            f"tại** vượt ngưỡng **{exception_threshold:.2f}** — gợi ý tham mưu chuyển lớp / phụ
đạo."
        )
        groups = result[group_col].unique().tolist()
        if len(groups) == 2:
            means_by_group = result.groupby(group_col)[score_col].mean()
            low_group, high_group = means_by_group.idxmin(), means_by_group.idxmax()
            exc_up = result[(result[group_col] == low_group) & (result["gamma_cum_2"] >
exception_threshold)]
            exc_down = result[(result[group_col] == high_group) & (result["gamma_cum_2"] < 1 -
exception_threshold)]

        cA, cB = st.columns(2)
        with cA:
            st.markdown(
                f"<div class='smartz-flag-up'> ▲ <b>{len(exc_up)} học sinh</b> thuộc '
                f"<i>{low_group}</i> có năng lực gần với cụm điểm cao ( $\gamma$  > '
                f'{exception_threshold:.2f}) — cân nhắc bồi dưỡng / đề xuất chuyển lớp.</div>",
                unsafe_allow_html=True,
            )
            st.dataframe(exc_up, use_container_width=True, height=220)
        with cB:
            st.markdown(
                f"<div class='smartz-flag-down'> ▼ <b>{len(exc_down)} học sinh</b> thuộc '
                f"<i>{high_group}</i> có năng lực gần với cụm điểm thấp ( $\gamma$  < '

```

```

        f'{1 - exception_threshold:.2f}') — cân nhắc phụ đạo thêm.</div>',
        unsafe_allow_html=True,
    )
    st.dataframe(exc_down, use_container_width=True, height=220)
else:
    st.info("Cần đúng 2 nhóm trong cột loại hình lớp để phát hiện ngoại lệ sự phạm.")
    idx_extra += 1
elif best_k > 2:
    with tabs[idx_extra]:
        st.caption(
            f"Phổ điểm được mô tả tốt nhất bởi {best_k} cụm năng lực. Bảng dưới đây cho thấy "
            f"xác suất (γ) mỗi học sinh thuộc về từng cụm — dùng để phân tích chi tiết thêm."
        )
        gamma_cols = [c for c in result.columns if c.startswith("gamma_cum_")]
        st.dataframe(result[[group_col, score_col] + gamma_cols], use_container_width=True)
        idx_extra += 1

with tabs[idx_extra]:
    st.dataframe(result, use_container_width=True)
    buf = io.BytesIO()
    with pd.ExcelWriter(buf, engine="openpyxl") as writer:
        result.to_excel(writer, index=False, sheet_name="KetQua")
    st.download_button(
        f"📄 Tải kết quả ({score_col}) — Excel", data=buf.getvalue(),
        file_name=f"ketqua_{score_col}.xlsx",
        mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        key=f"dl_{score_col}",
    )

```

```

def render_progress_comparison(raw: pd.DataFrame, group_col: str, results_by_col: dict,
col_from: str, col_to: str):
    """So sánh tiến bộ Z-score GMM Mềm giữa 2 cột điểm, ghép đúng theo học sinh (index gốc)."""
    r_from, r_to = results_by_col[col_from], results_by_col[col_to]
    if not (r_from["best_k"] >= 2 and r_to["best_k"] >= 2):
        st.info(
            "Chỉ có thể so sánh tiến bộ khi CẢ HAI cột điểm đều được xác định có từ 2 đỉnh trở lên "
            "(cột có 1 đỉnh dùng Z-score truyền thống, chưa có Z* để so sánh)."
        )
    return

merged = pd.concat(
    [raw[[group_col]], r_from["result"]["Z_GMM_mem"].rename(f"Z_GMM_mem_{col_from}")],

```



```

        r_to["result"]["Z_GMM_mem"].rename(f"Z_GMM_mem_{col_to}"),
        axis=1, join="inner",
    ).dropna()
    merged["Tien_bo"] = merged[f"Z_GMM_mem_{col_to}"] -
merged[f"Z_GMM_mem_{col_from}"]

    st.markdown(
        f"Chênh lệch Z-score GMM Mềm từ {col_from} sang {col_to} — dương nghĩa là tiến
bộ, "
        f"âm là sa sút, theo đúng nhóm năng lực tham chiếu của từng học sinh "
        f"(đã ghép đúng theo từng học sinh, n = {len(merged)})."
    )
    summary = merged.groupby(group_col)["Tien_bo"].agg(["mean", "std", "count"]).round(3)
    summary.columns = ["Tiến bộ TB", "Độ lệch chuẩn", "Số học sinh"]
    st.dataframe(summary, use_container_width=True)
    st.dataframe(merged, use_container_width=True)

```

```

# -----
# GIAO DIỆN CHÍNH
# -----

```

```

st.markdown(
    """
    <div class="smartz-hero">
        <h1>🎓 SmartZ-EDU</h1>
        <p>Hệ thống Z-score thích ứng bằng Mô hình Hỗn hợp Gauss Mềm (Soft GMM) —
tự động dò số đỉnh của phổ điểm (không áp đặt sẵn) để hỗ trợ đánh giá công bằng
và ra quyết định quản lý giáo dục.</p>
        <span class="smartz-badge">🧠 Soft GMM</span>
        <span class="smartz-badge">🔄 Tự động dò số đỉnh (k=1..4)</span>
        <span class="smartz-badge">🚩 Phát hiện ngoại lệ sự phạm</span>
        <span class="smartz-badge">📊 Theo dõi tiến bộ</span>
    </div>
    """,
    unsafe_allow_html=True,
)

```

```

with st.expander("📖 Về phương pháp Soft GMM và cơ chế thích ứng tự động"):
    st.markdown(
        """

```

- **Z-score GMM Mềm (phương pháp chính):** mỗi học sinh được gán trọng số xác suất γ (gamma) thuộc về từng cụm năng lực, thay vì bị ép buộc phân loại cứng.
- **Cơ chế thích ứng tự động (tổng quát):** hệ thống dò lần lượt mô hình với

k = 1, 2, 3, 4 thành phần Gauss, so sánh bằng tiêu chuẩn BIC, và chọn k có BIC nhỏ nhất — ****không áp đặt sẵn phải có đúng 2 đỉnh****. Nếu dữ liệu chỉ có 1 đỉnh, hệ thống tự quay về Z-score truyền thống; nếu có nhiều hơn 2 đỉnh, hệ thống vẫn tính được Z-score mềm tổng quát cho từng cụm.

""""

)

with st.sidebar:

st.markdown("### 📁 1. Tải dữ liệu")

uploaded = st.file_uploader("Chọn file Excel/CSV điểm số", type=["xlsx", "xls", "csv"])

st.caption(

"File cần có tối thiểu 2 cột: ****loại hình lớp**** (vd Lớp TC / Lớp hai buổi) và "

****điểm số****. Có thể có nhiều cột điểm (Giữa kỳ, Cuối kỳ...)."

)

st.markdown("---")

st.markdown("### ⚙️ 2. Tùy chọn phân tích")

k_max = st.slider(

"Số đỉnh tối đa để dò thử (k_max)", 2, 6, K_MAX_DEFAULT, 1,

help="Hệ thống sẽ tự động dò từ k=1 đến giá trị này và chọn k tối ưu theo BIC.",

)

exception_threshold = st.slider(

"Ngưỡng γ — 'ngoại lệ sự phạm' (áp dụng khi k=2)", 0.5, 0.9, 0.7, 0.05,

help="Học sinh có xác suất γ thuộc cụm khác vượt ngưỡng này sẽ được gán cờ.",

)

if uploaded is None:

st.info("👉 Hãy tải lên file dữ liệu điểm số ở thanh bên trái để bắt đầu.")

st.stop()

--- Đọc file, tự nhận diện dòng tiêu đề ---

try:

if uploaded.name.endswith(".csv"):

raw_no_header = pd.read_csv(uploaded, header=None)

else:

raw_no_header = pd.read_excel(uploaded, header=None)

except Exception as e:

st.error(f"Không đọc được file: {e}")

st.stop()

preview_rows = min(10, len(raw_no_header))

non_null_counts = raw_no_header.head(preview_rows).notna().sum(axis=1)

guess_header_row = int(non_null_counts.idxmax())

with st.expander("📄 3. Xem trước dữ liệu thô & chọn dòng tiêu đề", expanded=False):

```

st.dataframe(raw_no_header.head(preview_rows), use_container_width=True)
header_row = st.number_input(
    "Dòng nào là DÒNG TIÊU ĐỀ? (đánh số từ 0)",
    min_value=0, max_value=preview_rows - 1, value=guess_header_row, step=1,
)

raw = raw_no_header.iloc[header_row + 1:].copy()
raw.columns = raw_no_header.iloc[header_row].values
raw = raw.dropna(axis=1, how="all")
raw = raw.loc[:, [c for c in raw.columns if pd.notna(c)]]
raw.index = range(len(raw)) # chỉ số gốc dùng để ghép nối chính xác giữa các cột điểm

cols = raw.columns.tolist()
st.markdown("### 🎯 4. Chọn cột dữ liệu & chế độ phân tích")

c_group, c_mode = st.columns([1, 1])
with c_group:
    group_col = st.selectbox("Cột chứa LOẠI HÌNH LỚP", cols, index=0)
with c_mode:
    mode = st.radio(
        "Chế độ phân tích", ["🔍 Một cột điểm", "📊 Nhiều cột điểm (so sánh)"],
        horizontal=True,
    )

available_score_cols = [c for c in cols if c != group_col]

if mode == "🔍 Một cột điểm":
    score_cols = [st.selectbox("Chọn cột ĐIỂM SỐ cần phân tích", available_score_cols)]
else:
    score_cols = st.multiselect(
        "Chọn các cột ĐIỂM SỐ cần phân tích (vd Giữa kỳ và Cuối kỳ để so sánh tiến bộ)",
        available_score_cols, default=available_score_cols[:2],
    )

if not score_cols:
    st.warning("Vui lòng chọn ít nhất một cột điểm số.")
    st.stop()

st.markdown("---")

# --- Phân tích từng cột đã chọn ---
results_by_col = {}
for score_col in score_cols:
    r = analyze_column(raw, group_col, score_col, k_max=k_max)

```

```

if r is None:
    st.warning(f"Không có dữ liệu hợp lệ cho cột `{score_col}`.")
    continue
results_by_col[score_col] = r

if not results_by_col:
    st.stop()

if len(results_by_col) == 1:
    only_col = list(results_by_col.keys())[0]
    st.markdown(f"## 📊 Kết quả phân tích: `{only_col}`")
    render_analysis(raw, group_col, results_by_col[only_col], exception_threshold)
else:
    tab_labels = [f"📊 {c}" for c in results_by_col] + [f"📈 So sánh tiến bộ"]
    outer_tabs = st.tabs(tab_labels)
    for tab, score_col in zip(outer_tabs[:-1], results_by_col):
        with tab:
            render_analysis(raw, group_col, results_by_col[score_col], exception_threshold)
    with outer_tabs[-1]:
        keys = list(results_by_col.keys())
        cA, cB = st.columns(2)
        with cA:
            col_from = st.selectbox("Từ cột", keys, index=0)
        with cB:
            col_to = st.selectbox("Sang cột", keys, index=min(1, len(keys) - 1))
        if col_from == col_to:
            st.info("Hãy chọn hai cột điểm khác nhau để so sánh tiến bộ.")
        else:
            render_progress_comparison(raw, group_col, results_by_col, col_from, col_to)

st.markdown("---")
st.caption(
    "Phương pháp: hệ thống tự động dò số đỉnh ( $k=1..k_{max}$ ) của phổ điểm bằng tiêu chuẩn BIC, "
    "không áp đặt sẵn số cụm. Với  $k \geq 2$ , Z-score GMM Miền chuẩn hoá điểm theo trung bình có "
    "trọng số xác suất hậu nghiệm ( $\gamma$ ) giữa các thành phần Gauss, tránh phân loại cứng gây bất "
    "công ở vùng ranh giới."
)

```